

Chapter 4: Predictive Analytics

Supervised Machine Learning

Elmer Poliquit

Weeks 8 & 9, 2nd Semester, AY 2024 - 2025

Needed Packages

```
library(ggplot2)
library(caret)
library(class)      # For kNN
library(rpart)      # For decision trees
library(rpart.plot) # For plotting trees
library(shiny)      # For dashboard
library(plotly)     # For interactive plots
```

Introduction

- ▶ Supervised machine learning: learning a function from labeled data.
- ▶ Goal: predict the output (class or value) for new inputs.
- ▶ Focus of this chapter: classification, a type of supervised learning where output is categorical.

Distance-Based Classification

- ▶ Uses similarity or distance between data points.
- ▶ k-Nearest Neighbors (kNN) is a common method.

Mathematical Concept of kNN

Dataset:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$$

where

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip}) \quad , \quad y_i = \text{class label or numeric response}$$

Distance Metric (Euclidean):

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^p (x_j - x_{ij})^2}$$

k Nearest Neighbors:

$\mathcal{N}_k(\mathbf{x}_{\text{new}})$ = k points in the training set with smallest distance to $\mathbf{x}_{\text{new}}$

Prediction

Prediction (Classification)

$$\hat{y}_{\text{new}} = \text{mode}\{y_i \mid i \in \mathcal{N}_k(\mathbf{x}_{\text{new}})\}$$

Prediction (Regression):

$$\hat{y}_{\text{new}} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x}_{\text{new}})} y_i$$

Bias–Variance Tradeoff in kNN

- ▶ **Small k** $\rightarrow$ low bias, high variance (overfits noise)
- ▶ **Large k** $\rightarrow$ high bias, low variance (smooth prediction)
- ▶ kNN approaches the **Bayes optimal classifier** as:

$$n \rightarrow \infty \quad \text{and} \quad \frac{k}{n} \rightarrow 0$$

kNN Example in R

```
# Load dataset
data(iris)
set.seed(123)
train_index <- createDataPartition(iris$Species,
                                     p = 0.7, list = FALSE)

train <- iris[train_index, ]
test  <- iris[-train_index, ]
head(train)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa

kNN Example in R

Notes:

- ▶ k = number of neighbors
- ▶ Euclidean distance is default
- ▶ Works best with scaled/normalized data

```
head(test)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
16	5.7	4.4	1.5	0.4	setosa
18	5.1	3.5	1.4	0.3	setosa
20	5.1	3.8	1.5	0.3	setosa

kNN Example in R

```
options(width=50)
# Apply kNN
knn_pred <- knn(train[, 1:4], test[, 1:4],
                 train$Species, k = 3)
knn_pred
```

```
[1] setosa      setosa      setosa      setosa
[5] setosa      setosa      setosa      setosa
[9] setosa      setosa      setosa      setosa
[13] setosa      setosa      setosa      versicolor
[17] versicolor  versicolor  versicolor  versicolor
[21] versicolor  versicolor  versicolor  versicolor
[25] versicolor  versicolor  versicolor  versicolor
[29] versicolor  versicolor  virginica   virginica
[33] virginica   virginica   virginica   virginica
[37] versicolor  virginica   virginica   virginica
[41] versicolor  virginica   virginica   virginica
[45] virginica
Levels: setosa versicolor virginica
```


kNN Example in R

```
# Confusion matrix  
cM=confusionMatrix(knn_pred, test$Species)  
addmargins(cM$table)
```

Prediction	Reference			Sum
	setosa	versicolor	virginica	
setosa	15	0	0	15
versicolor	0	15	2	17
virginica	0	0	13	13
Sum	15	15	15	45

Setosa

- ▶ 15 correctly predicted as setosa
- ▶ 0 misclassified -> perfect classification

Versicolor

- ▶ 15 correctly predicted
- ▶ 2 misclassified as virginica

kNN Example in R

Prediction	Reference			Sum
	setosa	versicolor	virginica	
setosa	15	0	0	15
versicolor	0	15	2	17
virginica	0	0	13	13
Sum	15	15	15	45

Calculate overall accuracy

$$\text{Accuracy} = (\text{correct predictions}) / (\text{total samples})$$

$$\text{Accuracy} = \frac{15(\text{setosa}) + 15(\text{versicolor}) + 13(\text{virginica})}{15 + 15 + 2 + 13} \approx 95.6\%$$

So your k-NN model is approximately 95.6% accurate.

kNN Example in R

```
options(width=100)
# Confusion matrix
cM=confusionMatrix(knn_pred, test$Species)
cM$byClass
```

	Sensitivity	Specificity	Pos Pred Value	Neg Pred Value	Precision	Recall
Class: setosa	1.0000000	1.0000000	1.0000000	1.0000	1.0000000	1.0000000
Class: versicolor	1.0000000	0.9333333	0.8823529	1.0000	0.8823529	1.0000000
Class: virginica	0.8666667	1.0000000	1.0000000	0.9375	1.0000000	0.8666667
	F1	Prevalence	Detection Rate	Detection	Prevalence	Balanced Accuracy
Class: setosa	1.0000000	0.3333333	0.3333333	0.3333333	1.0000000	1.0000000
Class: versicolor	0.9375000	0.3333333	0.3333333	0.3777778	0.9666667	0.9666667
Class: virginica	0.9285714	0.3333333	0.2888889	0.2888889	0.9333333	0.9333333

Confusion Matrix Metrics Formulas

Suppose we have a **binary classification confusion matrix**:

	Predicted Positive	Predicted Negative
Actual Positive	True Positives	False Negatives
Actual Negative	False Positives	True Negatives

Confusion Matrix Metrics Formulas

Suppose we have a **binary classification confusion matrix**:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

1. Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Accuracy} = \frac{15(\text{setosa}) + 15(\text{versicolor}) + 13(\text{virginica})}{15 + 15 + 2 + 13} \approx 95.6\%$$

Confusion Matrix Metrics Formulas

Prediction	Reference			Sum
	setosa	versicolor	virginica	
setosa	15	0	0	15
versicolor	0	15	2	17
virginica	0	0	13	13
Sum	15	15	15	45

Sensitivity / Recall

- ▶ Probability that a sample of that class is correctly predicted.
Sensitivity=True Positives/(True Positives + False Negatives)
- ▶ Example: Versicolor = $\frac{15}{15+0} = 100\%$ of actual versicolor samples were correctly predicted.

kNN Example in R

Prediction	Reference			Sum
	setosa	versicolor	virginica	
setosa	15	0	0	15
versicolor	0	15	2	17
virginica	0	0	13	13
Sum	15	15	15	45

Specificity

- ▶ Probability that samples not in that class are correctly identified as not that class. $\text{Specificity} = \text{True Negatives} / (\text{True Negatives} + \text{False Positives})$
- ▶ Example: Versicolor specificity = $\frac{15+13}{(15+13)+2} = 93.333\%$ -> All non-versicolor samples were correctly not classified as versicolor.

Pos Pred Value / Precision

- ▶ Probability that a sample predicted as a class actually belongs to that class. $\text{Pos Pred Value} / \text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$
- ▶ Example: Versicolor precision = $\frac{15}{15+2} = 88.2\%$ of samples predicted as versicolor were truly versicolor.

kNN Example in R

	Sensitivity	Specificity	Pos Pred Value	Neg Pred Value	Precision	Recall
Class: setosa	1.0000000	1.0000000	1.0000000	1.0000	1.0000000	1.0000000
Class: versicolor	1.0000000	0.9333333	0.8823529	1.0000	0.8823529	1.0000000
Class: virginica	0.8666667	1.0000000	1.0000000	0.9375	1.0000000	0.8666667
	F1	Prevalence	Detection Rate	Detection	Prevalence	Balanced Accuracy
Class: setosa	1.0000000	0.3333333	0.3333333	0.3333333	0.3333333	1.0000000
Class: versicolor	0.9375000	0.3333333	0.3333333	0.3777778	0.3777778	0.9666667
Class: virginica	0.9285714	0.3333333	0.2888889	0.2888889	0.2888889	0.9333333

F1-score

- ▶ Harmonic mean of precision and recall -> balances false positives and false negatives. $F1\ Score = 2 \times (Precision \times Recall) / (Precision + Recall)$

Example: Versicolor $F1 = 2 \times \frac{0.8823529 \times 1}{0.8823529 + 1} = 0.9375$ -> good but slightly lower than setosa or virginica.

Metric	Value	Interpretation
Precision	0.9	90% of predicted positives are correct
Recall	0.6	Only 60% of actual positives are detected
F1 Score	0.72	Balanced measure: decent but recall is limiting performance

Source: S. Sastry et al., "Evaluation Metrics for Classification Problems," in Data Mining Concepts and Techniques, 3rd Edition, Elsevier, 2011.

Introduction

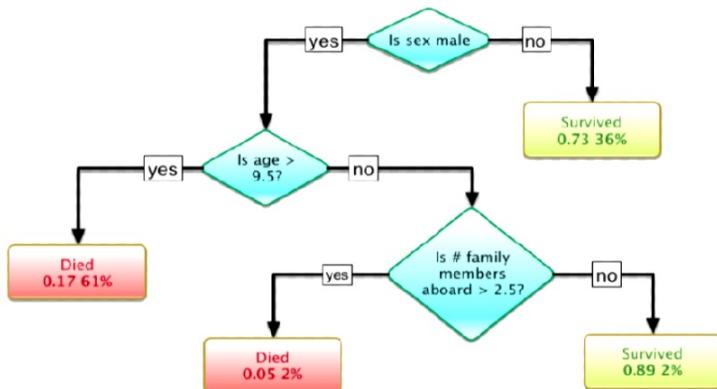
- ▶ Generalized Linear Models (GLM) extend the application of regression to various types of dependent variables.
- ▶ Principal Component Regression and Regularization methods provide tools for modeling high dimensional data and/or dimension reduction for modeling.
- ▶ Both can result to an interpretable model, and adequate prediction accuracy for several applications.
- ▶ A classification technique that performs well across a wide range of situations that is readily understandable by researchers is the tree methodology developed by Brieman, Friedman, Olshen, and Stone (1984).
- ▶ The program that Brieman et. al. created to implement these procedures was called **CART** for *Classification And Regression Trees*.

Tree-based Methods

- ▶ If the *response* variable is *continuous*, the tree is called a **regression tree**.
- ▶ If the *response* is *categorical*, the tree is called a **classification tree**.
- ▶ Both are collectively called classification and regression trees (**CART**).
- ▶ **CART** is one of the simplest and now most often used techniques for classification.
- ▶ The **CART** algorithm generates a classification tree by sequentially doing binary splits on the data.
- ▶ The simplest case is when splits are made on individual variables.

Introduction

Goal: Classify or predict an outcome based on a set of predictors where the output is a set of rules.



No higher resolution available.

[Titanic_Survival_Decision_Tree_SVG.png](#) (673 × 412 pixels, file size: 72 KB, MIME type: image/png)

Advantages and Disadvantages of Using Trees

- ▶ Trees are very easy to explain to people. In fact, they are even easier to explain than linear regression.
- ▶ Some people believe that decision trees closely mirror human decision-making.
- ▶ Trees can be displayed graphically and are easily interpreted even by a non-expert.
- ▶ Trees can easily handle qualitative predictors without the need to create dummy variables.
- ▶ Unfortunately, trees generally do not have the same level of predictive accuracy as some of the other regression and classification approaches.

Two Key Ideas Underlying Classification Trees

1. Recursive partitioning of the space of independent variables
 - ▶ Repeatedly split the records into two parts to achieve maximum homogeneity within the new parts
2. Pruning using validation data
 - ▶ Simplify the tree by pruning peripheral branches to avoid overfitting

The Tree Building Process

- ▶ In theory, the regions could have any shape. However, it is more convenient to divide the predictor space into *rectangles*, and such division results in interpretability and simplicity.
- ▶ For a given data, the goal is to find rectangles $R_1, R_2, \dots, R_j$ that results to minimum Residual Sum of Squares (**RSS**), given by

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{R}_j)^2 \quad (1)$$

where $\hat{R}_j$ is the predicted value in rectangle R_j .

The Tree Building Process

Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into J boxes.

Solution:

- ▶ We resort to a *Top-down, greedy approach*; Step-by-step splitting, or Recursive partitioning.
- ▶ The approach is top-down because it begins at the top of the tree and then successively splits the predictor space.
- ▶ It is greedy because at each step of the tree-building process, the best split is made at that particular step.

The Tree Building Process

Example: Riding Mowers (Johnson and Wichern) Data

- ▶ A riding-mower manufacturer would like to find a way of *classifying families in a city* into those that are likely to *purchase a riding mower* and those *who are not likely to buy one*.
- ▶ A pilot random sample of 12 owners and 12 non-owners in the city is undertaken.
- ▶ The independent variables here are Income (X_1 in 000s) and Lot Size (X_2 in 000s sqft).
- ▶ The categorical Y (*Standardized Residuals*, Regression) variable has two classes: owners and non-owners.
- ▶ The data are shown in table and then plotted below.

The Tree Building Process

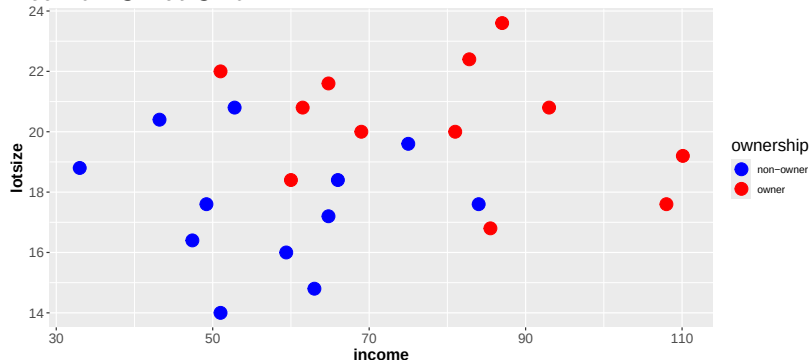
```
setwd("D:/Statistics Center/CART/Data")  
data=read.csv("buyers.csv")  
data
```

	income	lotsize	ownership
1	60.0	18.4	owner
2	85.5	16.8	owner
3	64.8	21.6	owner
4	61.5	20.8	owner
5	87.0	23.6	owner
6	110.1	19.2	owner
7	108.0	17.6	owner
8	82.8	22.4	owner
9	69.0	20.0	owner
10	93.0	20.8	owner
11	51.0	22.0	owner
12	81.0	20.0	owner
13	75.0	19.6	non-owner
14	52.8	20.8	non-owner
15	64.8	17.2	non-owner
16	43.2	20.4	non-owner
17	84.0	17.6	non-owner
18	49.2	17.6	non-owner
19	59.4	16.0	non-owner
20	66.0	18.4	non-owner
21	47.4	16.4	non-owner
22	33.0	18.8	non-owner
23	51.0	14.0	non-owner
24	63.0	14.8	non-owner

The Tree Building Process

Figure 1

Income VS Lot Size

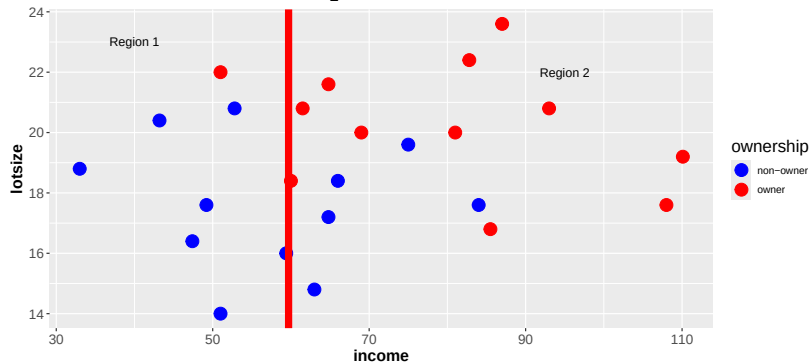


First, find the best predictor and best split. Find s_j for each X_j ($X_1 = \text{income}$) such that $X|X_j < s_j$ and $X|X_j \geq s_j$ results to the **best** RSS. The first split (s_1) is constructed on X_j with the minimum best RSS.

The Tree Building Process

Figure 2

Income VS Lot Size with s_1 , income = 59.7

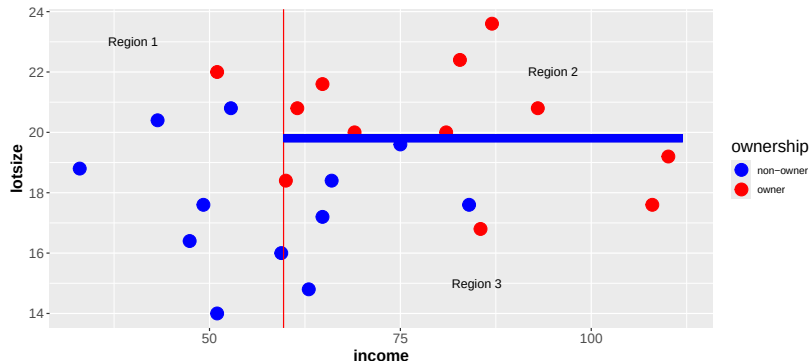


Consider $X_1 = \text{income} = 59.7$ and split within each region created in the previous step. Choose the best split based on the lowest RSS. Repeat the partitioning!

The Tree Building Process

Figure 3

Income VS Lot Size with s_2 , lotsize = 19.8

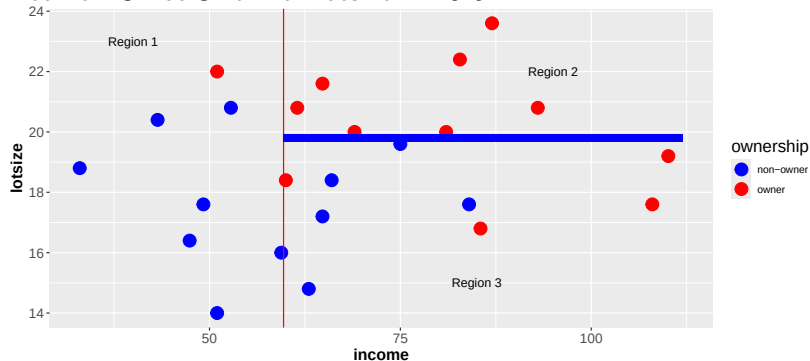


Consider ($X_2 = \text{lotsize} = 19.8$) and split within each region created in the previous step. The process stops until the change in RSS is too small ($< 5\%$), or until no region contains more than nine observations.

The Tree Building Process

Figure 3

Income VS Lot Size with $\text{lotsize} = 19.8$



In the case of sub-setting, we predict the response for a given test observation using the mean of the training observations in the region to which that test observation belongs.

The Tree Building Process

A **tree** is grown by binary recursive partitioning using the response in the specified formula and choosing splits from the terms of the right-hand side.

Preparing the data

```
library(tree)
data1=data.frame("income"=data$income,
                  "lotsize"=data$lotsize,
                  "ownership"=as.numeric(factor(data$ownership)))
head(data1)
```

	income	lotsize	ownership
1	60.0	18.4	2
2	85.5	16.8	2
3	64.8	21.6	2
4	61.5	20.8	2
5	87.0	23.6	2
6	110.1	19.2	2

The Tree Building Process

To use the *tree* function, the data must be in a data frame and the categorical variable is converted to numeric.

Splitting

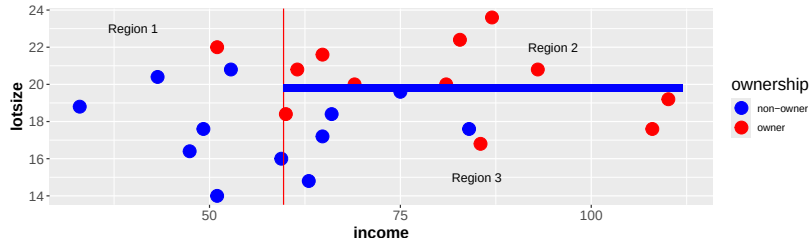
```
tree <- tree(ownership ~ income + lotsize, data = data1,  
            method = "recursive.partition",  
            split = c("deviance", "gini"))  
tree
```

```
node), split, n, deviance, yval  
    * denotes terminal node
```

- 1) root 24 6.000 1.500
- 2) income < 59.7 8 0.875 1.125 *
- 3) income > 59.7 16 3.438 1.688
- 6) lotsize < 19.8 9 2.222 1.444 *
- 7) lotsize > 19.8 7 0.000 2.000 *

The Tree Building Process

Splitting



node), split, n, deviance, yval
* denotes terminal node

- 1) root 24 6.000 1.500
- 2) income < 59.7 8 0.875 1.125 *
- 3) income > 59.7 16 3.438 1.688
- 6) lotsize < 19.8 9 2.222 1.444 *
- 7) lotsize > 19.8 7 0.000 2.000 *

The Tree Building Process

A **tree** is grown by binary recursive partitioning using the response in the specified formula and choosing splits from the terms of the right-hand side.

Regression Tree

```
summary(tree)
```

Regression tree:

```
tree(formula = ownership ~ income + lotsize, data = data1, method = "deviance", split = c("deviance", "gini"))
```

Number of terminal nodes: 3

Residual mean deviance: 0.1475 = 3.097 / 21

Distribution of residuals:

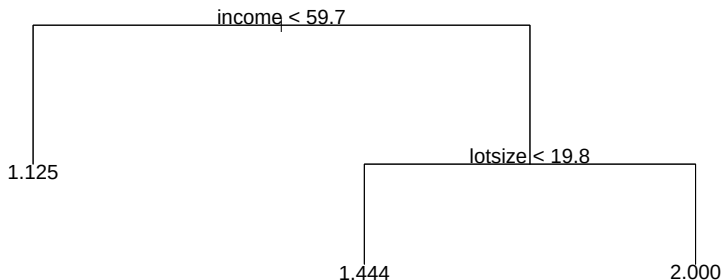
Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-0.4444	-0.1250	-0.0625	0.0000	0.0000	0.8750

The Tree Building Process

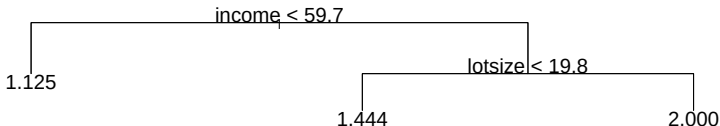
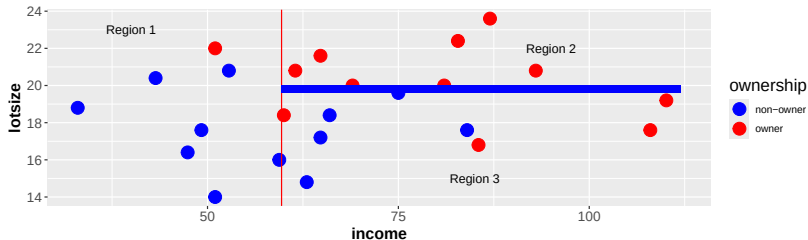
Tree

Number of terminal nodes: 3

```
plot(tree)  
text(tree,cex=1.5)
```



The Tree Building Process



Region 2 predicted 100% of **owner**.

Pruning a Tree

The process described above may produce good predictions on the training set, but is likely to overfit the data, leading to poor test set performance.

What if the greedy approach continued?

- ▶ A smaller tree with fewer splits (that is, fewer regions $R_1, R_2, \dots, R_j$) might lead to lower variance and better interpretation at the cost of a little bias. A tree with too many terminal nodes and/or regions may lead to overfitting.

Pruning a Tree

Cost Complexity Pruning

- ▶ Also known as *Weakest Link Pruning*.

Consider a sequence of trees indexed by a non-negative tuning parameter α . For each value of α there corresponds a subtree $T \subset T_0$ (large tree) such that

$$\sum_{m=1}^{|T|} \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

is as small as possible. Here $|T|$ indicates the number of terminal nodes of the tree T , R_m is the rectangle (i.e. the subset of predictor space) corresponding to the m^{th} terminal node, and $\hat{y}_{R_m}$ is mean in R_m .

Choosing the Best Tree

- ▶ The tuning parameter α controls a trade-off between the subtree's complexity and its fit to the training data.
- ▶ We select an optimal value $\hat{\alpha}$ using cross-validation).
- ▶ We then return to the full data set and obtain the subtree corresponding to the $\hat{\alpha}$.
- ▶ The *tree* function in R produces the number of terminal nodes.
- ▶ The *prune.tree* function has an option as to how many nodes will be produced.

Pruning a Tree

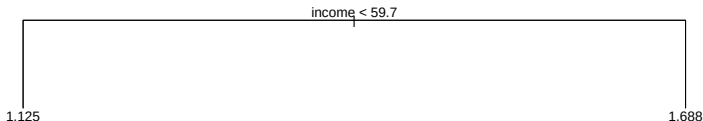
The tree shows the best 2 terminal nodes using *prune.tree*.

```
prunetree = prune.tree(tree, method = "deviance", best = 2)
prunetree
```

```
node), split, n, deviance, yval
    * denotes terminal node
```

```
1) root 24 6.000 1.500
  2) income < 59.7 8 0.875 1.125 *
  3) income > 59.7 16 3.438 1.688 *
```

```
plot(prunetree); text(prunetree)
```



Pruning a Tree

How many splits can be possible?

Examine all possible ways in which the categories can be split.

E.g., categories A , B , C can be split 3 ways

1. $\{A\}$ and $\{B, C\}$
2. $\{B\}$ and $\{A, C\}$
3. $\{C\}$ and $\{A, B\}$

With many categories, the number of splits becomes huge!

Why use the decision tree machine algorithm?

1. Easy to Understand and Interpret
2. Handles Different Types of Data
3. Minimal Data Preprocessing
4. Captures Non-Linear Relationships
5. Works Well for Classification Problems
6. Fast Training and Prediction
7. Feature Importance
8. Foundation for Advanced Models

Assumptions While Creating Decision Tree

- ▶ At the beginning, the *whole training set is considered as the root.*
- ▶ Feature values are preferred to be categorical. If the *values are continuous then they are discretized prior to building the model.*
- ▶ Records are *distributed recursively on the basis of attribute values.*
- ▶ Order to placing attributes as root or internal node of the tree is done by using some statistical approach.

Classification trees are designed for dependent variables that take a finite number of unordered values, with prediction error measured in terms of misclassification cost.

Misclassification costs are basically weights applied to specific outcomes.

Types of Measuring Impurity

1. The Gini Index

Suppose there are K different types of observations and $p_1, p_2, \dots, p_k$ are proportions each type in sample S . Then, the entropy of a sample S is computed as:

$$Entropy(S) = \sum_{i=1}^K p_i(1 - p_i).$$

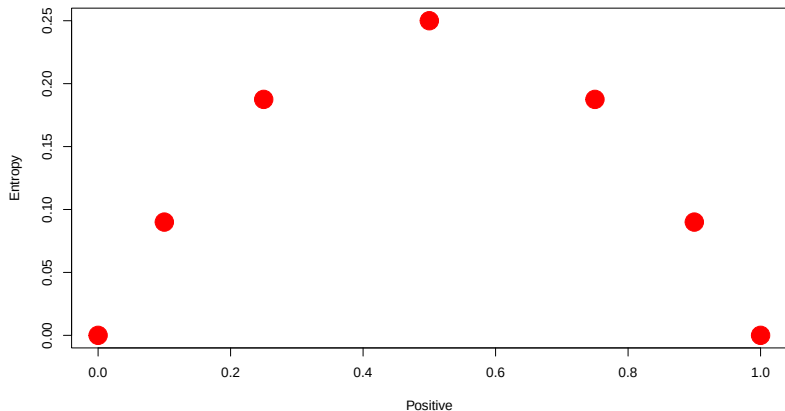
The *Entropy* is low when one type of observation tend to dominate the sample. Hence, low entropy means low impurity.

	Positive	Negative	Entropy
1	1.00	0.00	0.0000
2	0.90	0.10	0.0900
3	0.75	0.25	0.1875
4	0.50	0.50	0.2500
5	0.25	0.75	0.1875
6	0.10	0.90	0.0900
7	0.00	0.10	0.0000

Types of Measuring Impurity

1. The Gini Index

Illustration of Entropy for a binary data.



Types of Measuring Impurity

2. The Entropy-Based Measure

Entropy is a measure of impurity (with respect to predefined categories) within a sample.

Suppose there are K different types of observations and $p_1, p_2, \dots, p_k$ are proportions each type in sample S . Then, the entropy of a sample S is computed as:

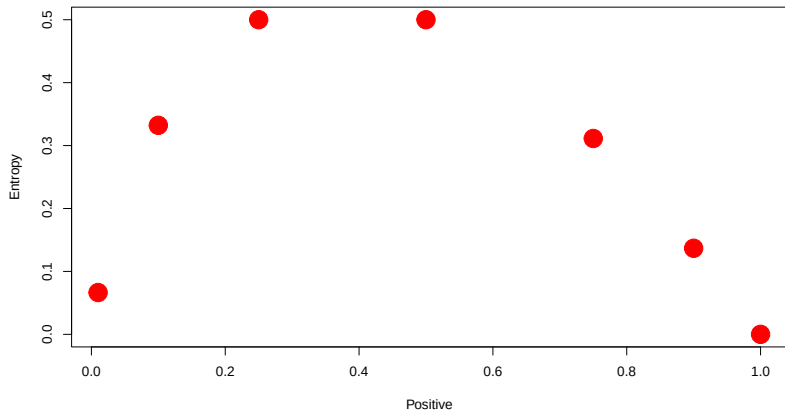
$$Entropy(S) = - \sum_{i=1}^K p_i (\log p_i).$$

	Positive	Entropy
1	1.00	0.00000000
2	0.90	0.13680278
3	0.75	0.31127812
4	0.50	0.50000000
5	0.25	0.50000000
6	0.10	0.33219281
7	0.01	0.06643856

Types of Measuring Impurity

2. The Entropy-Based Measure

Illustration of Entropy for a binary data.



Impurity and Recursive Partitioning

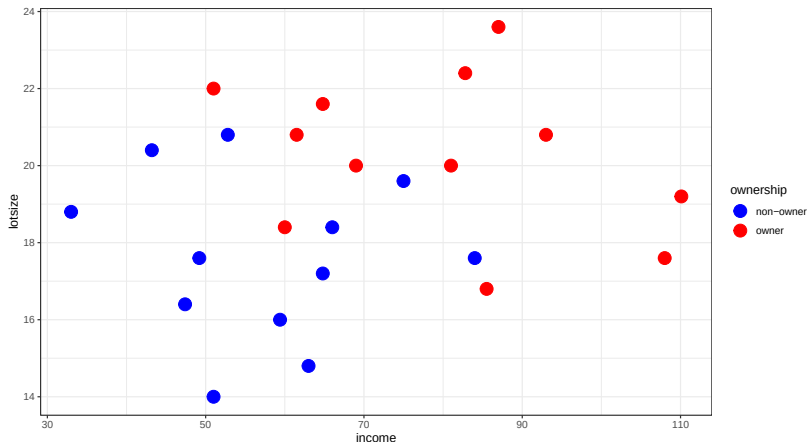
- ▶ Obtain overall impurity measure (weighted average of individual rectangles)
- ▶ At each successive stage, compare this measure across all possible splits in all variables
- ▶ Choose the split that reduces impurity the most
- ▶ Chosen split points become nodes on the tree

Entropy and Searching of the Next Partition

- ▶ Trees grow by doing recursive partitioning on the predictor space.
- ▶ The predictor space is repeatedly divided based on information gained, given previous divisions. For this to be possible, the following must be measured per iteration of the algorithm:
 - a. Entropy based response/target variable per region
 - b. Entropy after partitioning (weighted sum of entropy)
 - c. Decrease in Entropy

Recursive Partitioning based on Entropy

Suppose Y (owner/non-owner) is a binary variable, and X_1 (income) and X_2 (lotsize) are potential predictors of Y . We use Gini Index.



Recursive Partitioning based on Entropy

Suppose Y (owner/non-owner) is a binary variable, and X_1 (income) and X_2 (lotsize) are potential predictors of Y . We use Gini Index.

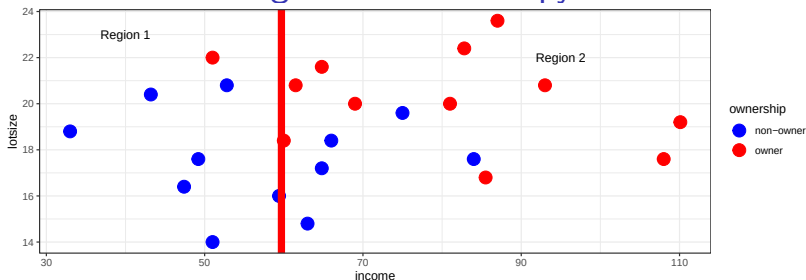
$$p(owner) = p(non - owner) = 0.5$$

The entropy based on Y :

$$Entropy(S) = 0.5(1 - 0.5) + 0.5(1 - 0.5)$$

	ownership	frequency	p_i	Entropy
1	owner	12	0.5	0.25
2	non-owner	12	0.5	0.25

Recursive Partitioning based on Entropy



The entropy after partitioning with respect to X_1 (income):

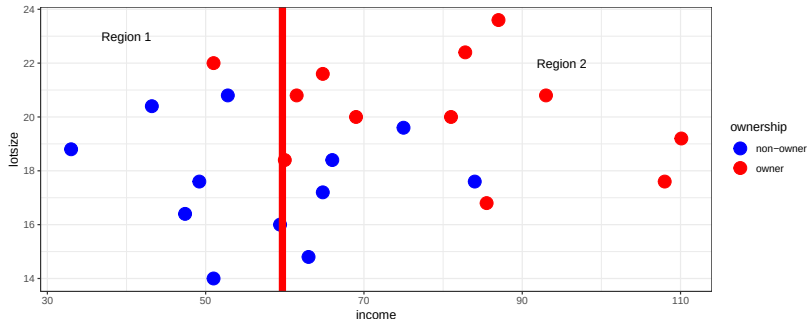
$$EntR_1 = \frac{1}{8} \times \left(1 - \frac{1}{8}\right) + \frac{7}{8} \times \left(1 - \frac{7}{8}\right) = 0.21875$$

$$EntR_2 = \frac{11}{16} \times \left(1 - \frac{11}{16}\right) + \frac{5}{16} \times \left(1 - \frac{5}{16}\right) = 0.42969$$

$$EntTree = p(R_1) \times EntR_1 + p(R_2) \times EntR_2 =$$

$$0.5 \times 0.21875 + 0.5 \times 0.42969 = 0.32422$$

Recursive Partitioning based on Entropy



The entropy after partitioning with respect to X_1 (income):

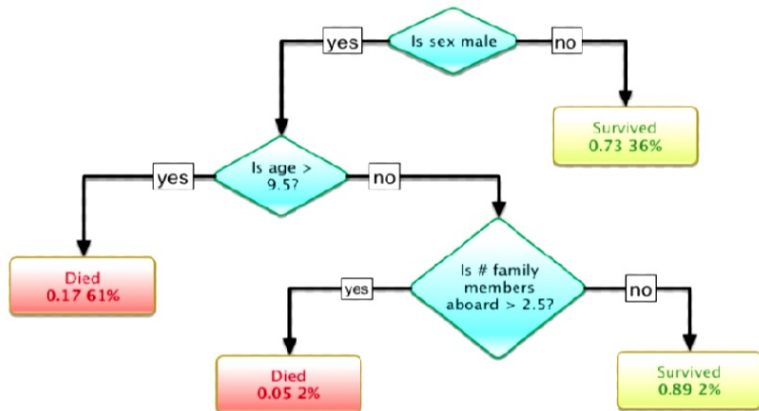
$EntR_1 = 0.21875$, $EntR_2 = 0.42969$, and $EntTree = 0.32422$

The goal of tree growing algorithms is to find the partition resulting to the **largest decrease** in entropy.

Then partition the region with the largest decrease in entropy.

Types of Measuring Impurity

In both cases the index is a function of the ratio between the categories of the observation in that node.



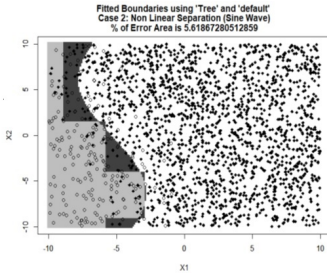
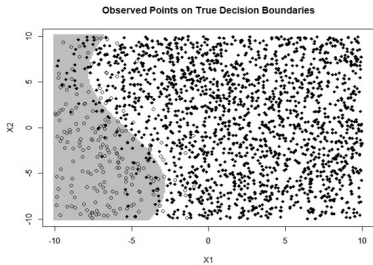
No higher resolution available.

[Titanic_Survival_Decision_Tree_SVG.png](#) (673 × 412 pixels, file size: 72 KB, MIME type: image/png)

Determining Leaf Node Label

- ▶ Each leaf node label is determined by *voting* of the records within it, and by the cutoff value
- ▶ Records within each leaf node are from the training data
- ▶ Default cutoff = 0.5 means that the leaf node's label is the majority class.
- ▶ Cutoff = 0.75: requires majority of 75% or more "1" records in the leaf to label it a "1" node

Decision Boundaries of Trees: A single tree may not generally fit well on data. Most especially if the true decision boundary is curvilinear.



Summary of the Top-Down Greedy Process with Entropy

1. Start with all data $\rightarrow$ root node.
2. For each feature, evaluate all possible splits.
3. Compute entropy or Gini for splits.
4. Pick the split that maximizes information gain / purity.
5. Split node and repeat recursively.
6. Stop when nodes are pure or other stopping criteria are met.

“Greedy” = always choose the best split at the current node, without considering future splits.

Decision Tree

Advantages

- ▶ Easy to use, understand
- ▶ Produce rules that are easy to interpret & implement
- ▶ Variable selection & reduction is automatic
- ▶ Do not require the assumptions of statistical models
- ▶ Can work without extensive handling of missing data

Disadvantages

- ▶ May suffer from overfitting
- ▶ Classifies by rectangular partitioning
- ▶ Does not easily handle nonnumeric data
- ▶ Can be quite large-pruning is necessary
- ▶ May not perform well where there is structure in the data that is not well captured by horizontal or vertical splits

Using rpart

Packages

```
library(rpart)
library(rpart.plot)
library(rattle)
```

Using rpart

- ▶ `rpart(formula, data, weights, subset, na.action = na.rpart, method, model = FALSE, x = FALSE, y = TRUE, parms, control, cost, ...)`

a `formula`, with a response but no interaction terms.

Model frame:

```
rpart(ownership ~ income + lotsize, data = data)
```

n= 24

node), split, n, loss, yval, (yprob)

* denotes terminal node

- 1) root 24 12 non-owner (0.5000000 0.5000000)
- 2) income< 59.7 8 1 non-owner (0.8750000 0.1250000) *
- 3) income>=59.7 16 5 owner (0.3125000 0.6875000) *

Using rpart

- ▶ `rpart(formula, data, weights, subset, na.action = na.rpart, method, model = FALSE, x = FALSE, y = TRUE, parms, control, cost, ...)`

an optional `data` frame in which to interpret the variables named in the formula

Model frame:

```
rpart(ownership ~ income + lotsize, data = data)
```

`n= 24`

```
node), split, n, loss, yval, (yprob)
      * denotes terminal node
```

```
1) root 24 12 non-owner (0.5000000 0.5000000)
   2) income< 59.7 8 1 non-owner (0.8750000 0.1250000) *
   3) income>=59.7 16 5 owner (0.3125000 0.6875000) *
```

Using rpart

- ▶ `rpart(formula, data, weights, subset, na.action = na.rpart, method, model = FALSE, x = FALSE, y = TRUE, parms, control, cost, ...)`

optional case **weights**

You may refer below.

<https://stackoverflow.com/questions/22258739/apply-weights-in-rpart-model-gives-error>

- ▶ `rpart(formula, data, weights, subset, na.action = na.rpart, method, model = FALSE, x = FALSE, y = TRUE, parms, control, cost, ...)`

optional expression saying that only a **subset** of the rows of the data should be used in the fit.

- Explicitly pick rows of your data.frame: **subset=1:21**
- Pick rows based on variable(s) value(s): **subset=(Age<50)**

Using rpart

- ▶ `rpart(formula, data, weights, subset, na.action = na.rpart, method, model = FALSE, x = FALSE, y = TRUE, parms, control, cost, ...)`

na.action

the default action deletes all observations for which y is **missing** (`na.omit`), but keeps those in which one or more predictors are missing. (`na.pass` does nothing)

Using rpart

- ▶ `rpart(formula, data, weights, subset, na.action = na.rpart, method, model = FALSE, x = FALSE, y = TRUE, parms, control, cost, ...)`
- a. If `method` is missing then the routine tries to make an intelligent guess.
- b. If `y` is a survival object, then `method` = “exp” is assumed.
- c. If `y` has 2 columns then `method` = “poisson” is assumed.
- d. If `y` is a factor then `method` = “class” is assumed, otherwise `method` = “anova” is assumed.
- e. It is wisest to specify the `method` directly, especially as more criteria may added to the function in future.
- f. Alternatively, `method` can be a list of functions named `init`, `split` and `eval`.

Using rpart

- ▶ `rpart(formula, data, weights, subset, na.action = na.rpart, method, model = FALSE, x = FALSE, y = TRUE, parms, control, cost, ...)`

if logical: keep a copy of the model frame in the result (TRUE or FALSE)? If the input value for `model` is a model frame (likely from an earlier call to the `rpart` function), then this frame is used rather than constructing new data

- ▶ `rpart(formula, data, weights, subset, na.action = na.rpart, method, model = FALSE, x = FALSE, y = TRUE, parms, control, cost, ...)`

`x` - keep a copy of the `x` matrix in the result.

`y` - keep a copy of the dependent variable in the result. If missing and `model` is supplied this defaults to FALSE.

Using rpart

- ▶ `rpart(formula, data, weights, subset, na.action = na.rpart, method, model = FALSE, x = FALSE, y = TRUE, parms, control, cost, ...)`

parms - optional parameters for the splitting function.

- The Anova splitting has no parameters.
- The Poisson splitting has a single parameter, the coefficient of variation of the prior distribution on the rates. The default value is 1.
- Exponential splitting has the same parameter as Poisson.
- For classification splitting, the list can contain any of: the vector of prior probabilities (component prior), the loss matrix (component loss) or the splitting index (component split). The priors must be positive and sum to 1.
- The loss matrix must have zeros on the diagonal and positive off-diagonal elements. The splitting index can be gini or information. The default priors are proportional to the data counts, the losses default to 1, and the split defaults to gini.

Using rpart

Example:

```
rpart(Kyphosis ~ Age + Number + Start,  
      data = kyphosis,  
      parms = list(prior = c(0.65,0.35),  
                   split = "information"))
```

n= 81

node), split, n, loss, yval, (yprob)
* denotes terminal node

- 1) root 81 28.350000 absent (0.65000000 0.35000000)
- 2) Start>=12.5 46 3.335294 absent (0.91563089 0.08436911) *
- 3) Start< 12.5 35 16.453120 present (0.39676840 0.60323160)
- 6) Age< 34.5 10 1.667647 absent (0.81616742 0.18383258) *
- 7) Age>=34.5 25 9.049219 present (0.27932897 0.72067103) *

Using rpart

```
rpart(formula, data, weights, subset, na.action = na.rpart,  
method}, model = FALSE, x = FALSE, y = TRUE, parms,  
control, cost, ...)
```

control - a list of options that control details of the rpart algorithm.

Example

```
rpart.control(minsplit = 20, minbucket = round(minsplit/3), cp =  
0.01, maxcompete = 4, maxsurrogate = 5, usesurrogate = 2, xval  
= 10, surrogatestyle = 0, maxdepth = 30, ...)
```


Using rpart

```
rpart.control(minsplit = 20, minbucket = round(minsplit/3), cp =  
0.01, maxcompete = 4, maxsurrogate = 5, usesurrogate = 2, xval  
= 10, surrogatestyle = 0, maxdepth = 30, ...)
```

cp - complexity parameter.

- ▶ Any split that does not decrease the overall lack of fit by a factor of **cp** is not attempted.
- ▶ For instance, with anova splitting, this means that the overall R-squared must increase by **cp** at each step.
- ▶ The main role of this parameter is to save computing time by pruning off splits that are obviously not worthwhile. - Essentially, the user informs the program that any split which does not improve the fit by **cp** will likely be pruned off by cross-validation, and that hence the program need not pursue it.

The arguments to **rpart.control** may also be specified in the call to **rpart**. They are checked against the list of valid arguments.

Using rpart

```
rpart(formula, data, weights, subset, na.action = na.rpart,  
method}, model = FALSE, x = FALSE, y = TRUE, parms,  
control, cost, ...)
```

cost - a vector of non-negative costs, one for each variable in the model.

- Defaults to one for all variables. These are scaling to be applied when considering splits, so the improvement on splitting on a variable is divided by its cost in deciding which split to choose.

Using rpart

Example

```
rpart=rpart(ownership ~ income + lotsize, data = data,  
            method = "class")  
names(rpart)
```

[1] "frame"	"where"	"call"
[5] "cptable"	"method"	"parms"
[9] "functions"	"numresp"	"splits"
[13] "y"	"ordered"	

Using rpart

```
summary(rpart)
```

Call:

```
rpart(formula = ownership ~ income + lotsize, data = data, method = "class")
n= 24
```

	CP	nsplit	rel error	xerror	xstd
1	0.50	0	1.0	1.416667	0.1855610
2	0.01	1	0.5	1.083333	0.2034141

Variable importance

income	lotsize
80	20

Node number 1: 24 observations, complexity param=0.5
predicted class=non-owner expected loss=0.5 P(node) =1
class counts: 12 12
probabilities: 0.500 0.500
left son=2 (8 obs) right son=3 (16 obs)

Primary splits:

income < 59.7 to the left, improve=3.375000, (0 missing)
lotsize < 19.8 to the left, improve=3.085714, (0 missing)

Surrogate splits:

lotsize < 16.6 to the left, agree=0.75, adj=0.25, (0 split)

Node number 2: 8 observations

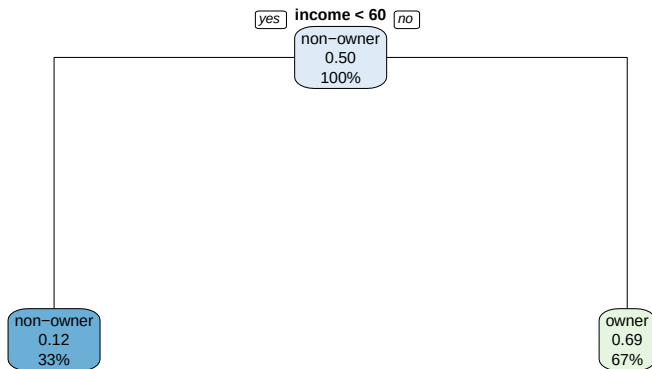
predicted class=non-owner expected loss=0.125 P(node) =0.3333333
class counts: 7 1
probabilities: 0.875 0.125

Node number 3: 16 observations

predicted class=owner expected loss=0.3125 P(node) =0.6666667
class counts: 5 11
probabilities: 0.312 0.688

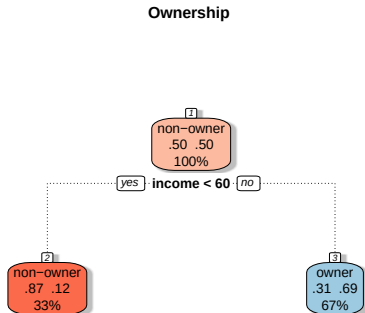
Using rpart

```
rpart.plot(rpart, type = 1, cex = 1.2)
```



Using rpart

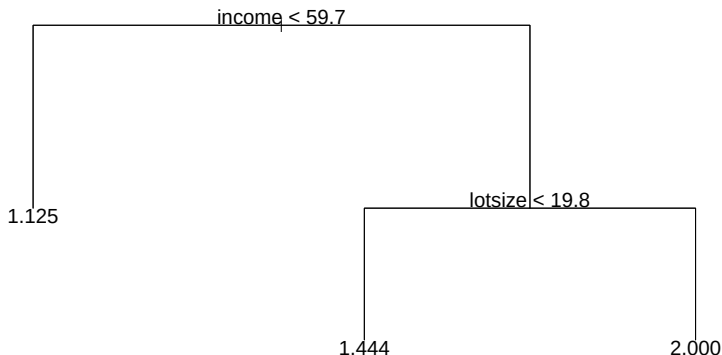
```
fancyRpartPlot(rpart, main = "Ownership", type = 2,  
               palettes = c("Reds", "Blues"))
```



Using rpart

Tree

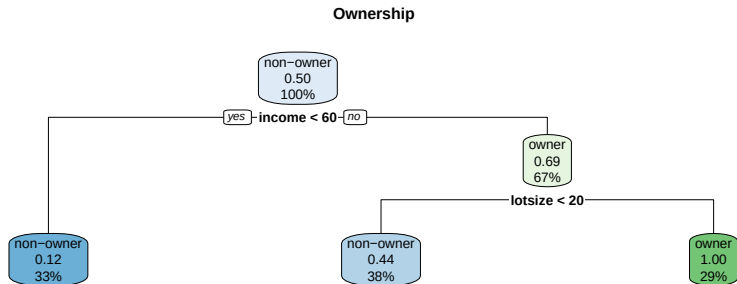
```
plot(tree)  
text(tree,cex=1.5)
```



Using rpart

Same graph with the tree

```
rpart1 <- rpart(ownership ~ income + lotsize, data = data,  
  method = "class", minsplit = 10) #Minimum split - 10 each  
rpart.plot(rpart1, main = "Ownership", type = 2); 33+38+29
```

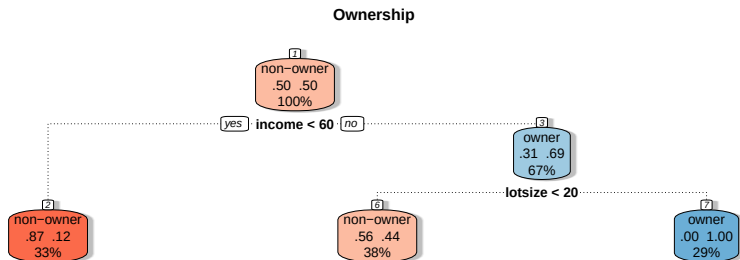


[1] 100

Using rpart

Same graph with the tree

```
fancyRpartPlot(rpart1, main = "Ownership", type = 2,  
  palettes = c("Reds", "Blues"))
```



Rattle 2026-Jan-18 18:01:34 ASUS TUF GAMING

Using rpart

Predict

Suppose that we want to predict an income from 55 to 65 and a lotsize from 16 to 26.

```
options(width = 50)
newdata = data.frame(income = 55:65, lotsize = 16:26)
predict(rpart, newdata, type = 'class')
```

1	2	3	4	5
non-owner	non-owner	non-owner	non-owner	non-owner
6	7	8	9	10
owner	owner	owner	owner	owner
11				
owner				

Levels: non-owner owner

Using rpart

Predict - Using Train Data

```
t = table(actual = data$ownership,  
          predicted = predict(rpart,  
                             newdata1 = data.frame(income = data$income,  
                                                    lotsize = data$lotsize), type = 'class'))  
addmargins(t)
```

	predicted		
actual	non-owner	owner	Sum
non-owner	7	5	12
owner	1	11	12
Sum	8	16	24

```
addmargins(round(prop.table(t),2))
```

	predicted		
actual	non-owner	owner	Sum
non-owner	0.29	0.21	0.50
owner	0.04	0.46	0.50
Sum	0.33	0.67	1.00

Another example

Iris Data

```
options(width = 100)
data(iris)
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

```
dim(iris)
```

```
[1] 150  5
```

Another example

```
table(iris$Species)
```

```
setosa versicolor virginica  
    50         50         50
```

```
tree1=tree(Species~., data = iris, method = "class",  
           split = c("deviance","gini"))  
summary(tree1)
```

Classification tree:

```
tree(formula = Species ~ ., data = iris, method = "class", split = c("deviance"  
  "gini"))
```

Variables actually used in tree construction:

```
[1] "Petal.Length" "Petal.Width" "Sepal.Length"
```

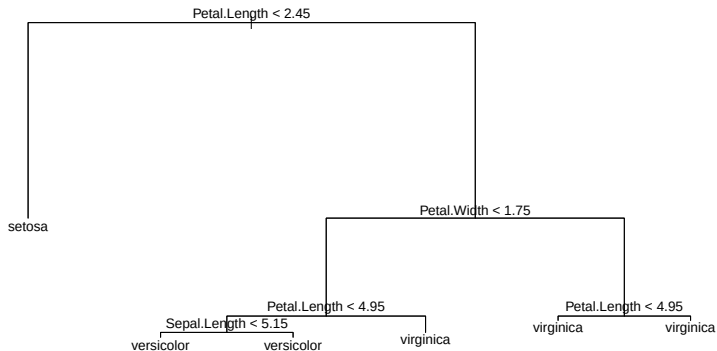
Number of terminal nodes: 6

Residual mean deviance: 0.1253 = 18.05 / 144

Misclassification error rate: 0.02667 = 4 / 150

Another example

```
plot(tree1)  
text(tree1)
```



Another example

Pruning a tree

```
prunetree1 = prune.tree(tree1, best = 3, method = "deviance")
print(prunetree1)
```

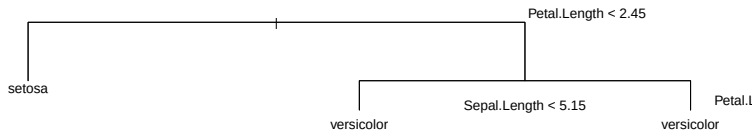
```
node), split, n, deviance, yval, (yprob)
      * denotes terminal node
```

```
1) root 150 329.600 setosa ( 0.33333 0.33333 0.33333 )
  2) Petal.Length < 2.45 50 0.000 setosa ( 1.00000 0.00000 0.00000 ) *
  3) Petal.Length > 2.45 100 138.600 versicolor ( 0.00000 0.50000 0.50000 )
    6) Petal.Width < 1.75 54 33.320 versicolor ( 0.00000 0.90741 0.09259 ) *
    7) Petal.Width > 1.75 46 9.635 virginica ( 0.00000 0.02174 0.97826 ) *
```

Another example

Pruning a tree

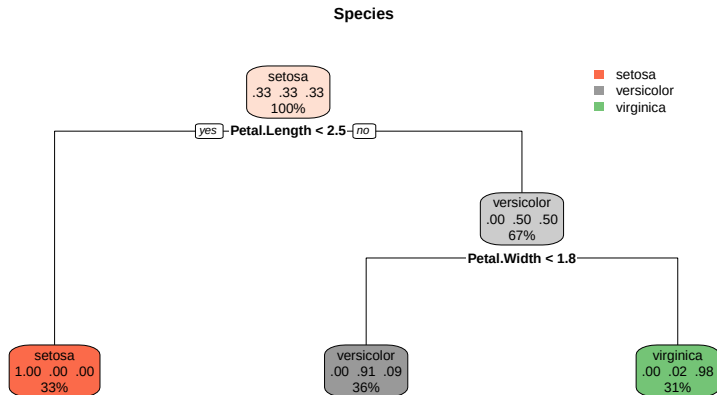
```
plot(prunetree1)  
text(tree1)
```



Another example

Pruning a tree

```
rpart = rpart(Species~., data = iris, method = "class",  
              minsplit = 50)  
rpart.plot(rpart, type = 2, cex = 1, main = "Species")
```



Another example

Predict

```
table = table(actual = iris$Species,  
              predicted = predict(rpart,  
                                newdata1 = data.frame(Sepal.Length=iris$Sepal.Length,  
                                                       Sepal.Width=iris$Sepal.Width,  
                                                       Petal.Length=iris$Petal.Length,  
                                                       Petal.Width=iris$Petal.Width), type = 'class'))  
addmargins(table)
```

	predicted			
actual	setosa	versicolor	virginica	Sum
setosa	50	0	0	50
versicolor	0	49	1	50
virginica	0	5	45	50
Sum	50	54	46	150

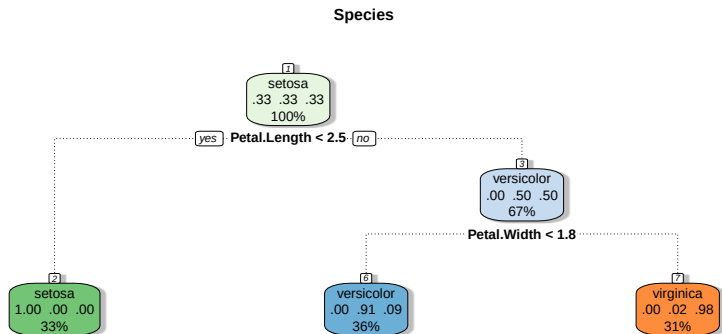
```
addmargins(round(prop.table(table),2))
```

	predicted			
actual	setosa	versicolor	virginica	Sum
setosa	0.33	0.00	0.00	0.33
versicolor	0.00	0.33	0.01	0.34
virginica	0.00	0.03	0.30	0.33
Sum	0.33	0.36	0.31	1.00

Another example

Pruning a tree

```
fancyRpartPlot(rpart, type = 2, cex = 1, main = "Species")
```



Rattle 2026-Jan-18 18:01:36 ASUS TUF GAMING

Dashboard Implementation of Classification Models

Packages Needed

```
library(shiny)
library(plotly)
library(rpart)
data(iris)
model <- rpart(Species ~ ., iris, method = "class")
```

Dashboard Implementation of Classification Models

```
ui <- fluidPage(  
  titlePanel("Iris Classifier"),  
  sidebarLayout(  
    sidebarPanel(  
      numericInput("sl", "Sepal Length", 5),  
      numericInput("sw", "Sepal Width", 3.5),  
      numericInput("pl", "Petal Length", 1.4),  
      numericInput("pw", "Petal Width", 0.2),  
      actionButton("go", "Predict")  
    ),  
    mainPanel(  
      textOutput("pred"),  
      plotlyOutput("plot")  
    )  
  )  
)  
  
server <- function(input, output) {  
  observeEvent(input$go, {  
    x <- data.frame(  
      Sepal.Length=input$sl,  
      Sepal.Width=input$sw,  
      Petal.Length=input$pl,  
      Petal.Width=input$pw  
    )  
    output$pred <- renderText(  
      paste("Predicted Species:", predict(model, x, type="class"))  
    )  
    output$plot <- renderPlotly(  
      plot_ly(iris, x=~Sepal.Length, y=~Petal.Length,  
        color=~Species, type="scatter", mode="markers") %>%  
        add_markers(x=input$sl, y=input$pl, inherit=FALSE)  
    )  
  })  
}
```

Dashboard Implementation of Classification Models

```
#shinyApp(ui, server)
```

Activity for kNN and CART

Learning Objectives

By the end of this activity, students should be able to:

- ▶ Apply k-Nearest Neighbors (kNN) for classification
- ▶ Build a CART (Decision Tree) model
- ▶ Preprocess real-world data
- ▶ Evaluate models using a confusion matrix
- ▶ Compare distance-based and tree-based classifiers

Data

- ▶ `install.packages("titanic")`
- ▶ `library(titanic)`
- ▶ `data(titanic_train)`

```
titanic <- read.csv(  
  "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"  
)
```

Activity for kNN and CART

Dataset: Titanic Survival Data

```
library(titanic)
data(titanic_train)
titanic <- titanic_train[,
  c("Survived", "Pclass", "Sex", "Age", "Fare")]
titanic <- na.omit(titanic)
str(titanic)
```

```
'data.frame':  714 obs. of  5 variables:
 $ Survived: int  0 1 1 1 0 0 0 1 1 1 ...
 $ Pclass  : int  3 1 3 1 3 1 3 3 2 3 ...
 $ Sex     : chr  "male" "female" "female" "female" ...
 $ Age     : num  22 38 26 35 35 54 2 27 14 4 ...
 $ Fare    : num  7.25 71.28 7.92 53.1 8.05 ...
 - attr(*, "na.action")= 'omit' Named int [1:177] 6 18 20 27 29 30 32 3
 ..- attr(*, "names")= chr [1:177] "6" "18" "20" "27" ...
```

```
titanic$Survived <- factor(titanic$Survived)
titanic$Sex <- factor(titanic$Sex)
```


Activity for kNN and CART

Use the same *set.seed*

1. Perform kNN
 - a. Why does accuracy drop when k becomes too large?
 - b. Why is scaling required for kNN?
 - c. What is the trade-off between small and large k ?
2. Perform CART (Decision Tree Classification)
 - a. Which tree is easier to interpret?
 - b. Which model performs better on test data?
 - c. Which tree is more likely to overfit?

Submission Requirement: Knit your RMarkdown file to HTML and upload the resulting file to Canvas. Use the same title file.

References

- ▶ Downey, A. B. (2014). Think Stats: Exploratory Data Analysis (2nd ed.). O'Reilly Media / Green Tea Press.
- ▶ Golemund, G., & Wickham, H. (2017). R for data science: Import, tidy, transform, visualize, and model data. O'Reilly Media.
- ▶ Wickham, H., & Golemund, G. (2023). R for Data Science: Import, Tidy, Transform, Visualize, and Model Data (2nd ed.)
- ▶ James, G. et al., An Introduction to Statistical Learning, 2021
- ▶ Tan, Steinbach, Kumar, Introduction to Data Mining, 2022
- ▶ R Documentation: kmeans, dbSCAN, hclust, arules

THANK YOU!